# Raspberry Pi Zero 2 W — running one test on the board, step by step

This page runs one µOS++ test on a Raspberry Pi Zero 2 W through OpenOCD, by hand, for the 32-bit (AArch32) and the 64-bit (AArch64) port. It does the same thing the test scripts (`hw.sh`, `run-hw.sh`) do, written out as plain commands.

The probe is a SEGGER J-Link, used through OpenOCD. (An Olimex ARM-USB-OCD also works; see the end of the page.)

## What happens, in short

1. The board is reset through its watchdog, and boots again from its SD card. Core 0 runs the SD card's kernel; cores 1–3 wait in the firmware's own loop.
2. OpenOCD stops the four CPU cores.
3. OpenOCD turns semihosting on, so the test can print through the probe.
4. OpenOCD copies the test program into the board's RAM.
5. OpenOCD starts **core 0** at the program's first instruction, and lets cores 1–3 go on where they were: still waiting in the firmware's loop.
6. When the test's kernel is ready, it releases cores 1–3 itself.
7. The test runs and prints `RESULT: PASS` (or `FAIL`).

Each test starts with the watchdog reset of step 4, so the tests can be run one after another. (On 2026-10-09 all 14 tests but `usb_test` passed this way, back to back, on both ports.) If a test hangs, power-cycle the board.

## The SD card

Cores 1–3 must still be in the firmware's loop when step 6 runs, so the SD card must boot a kernel that does **not** start them, and it must boot the same width as the port, because the two ports wake the cores differently:

| Port | SD card `config.txt` | The kernel wakes cores 1–3 by writing | Cores 1–3 wait at (after step 4) |
|---|---|---|---|
| AArch32 | `arm_64bit=0` | their mailbox 3 | `pc=0x7a`, Thumb, Hypervisor |
| AArch64 | `arm_64bit=1` (e.g. `$A64/test/boards/rpi-zero-2w/hw-park/`) | their release word at `0xd8 + 8*core` | `pc=0x7c`, EL2H |

So switch the SD card (or its `config.txt` and kernel) when you switch ports. To check it, after a step-4 reset and a 12 s wait:

```
$OPENOCD -s $CFG -f $CFG/openocd-jlink-rpi3.cfg -c "init" \
  -c "targets bcm2837.cpu1" -c "halt" -c "reg pc" -c "resume" \
  -c "shutdown"
```

`pc` must be the value in the table (the same for `cpu2`, `cpu3`). If a core is anywhere else, the SD card's kernel has started it, and these steps will not work: use another card, or the previous flow, kept in each port's `test/boards/rpi-zero-2w/hw.sh.bak` (it starts all four cores and first clears the `__smp_spin` table in RAM).

## Step 0 — set the paths

```
WORK=/home/dan/Desktop/tmp_tests          # the folder that holds the repositories
K=$WORK/micro-os-plus-iii                 # kernel
A32=$WORK/micro-os-plus-iii-aarch32       # 32-bit port
A64=$WORK/micro-os-plus-iii-aarch64       # 64-bit port
BUILD=$K/tests/build
OPENOCD=$HOME/.local/xPacks/@xpack-dev-tools/openocd/0.12.0-7.1/.content/bin/openocd
```

## Step 1 — build the tests

```
cd $K/tests
xpm install     --config aarch32-rpi-zero-2w-cmake-gcc-debug
xpm run prepare --config aarch32-rpi-zero-2w-cmake-gcc-debug
xpm run build   --config aarch32-rpi-zero-2w-cmake-gcc-debug
```

For the 64-bit port, write `aarch64` instead of `aarch32`.

The programs for the board are the files ending in `-hwd`, in `$BUILD/aarch32-rpi-zero-2w-cmake-gcc-debug/platform-bin/port-tests/test/` (or `aarch64-...`). The tests are: `smp_test0` … `smp_test4`, `smp-mat-test`, `smp-mat-sdcard-test`, `smp-num-test`, `smp-pipeline-test`, `smp-pro-cons-test`, `sd_test`, `usb_test`, `cmsis-os-validator`, `mutex-stress`, `rtos-apis`.

---

# AArch32 (32-bit port)

### Step 2 — choose the test

```
D=$BUILD/aarch32-rpi-zero-2w-cmake-gcc-debug/platform-bin/port-tests/test
CFG=$A32/test/boards/rpi-zero-2w
ELF=$D/smp_test0-hwd
```

Nothing has to be read from the ELF: the program always starts at `0x1003c`.

### Step 3 — open the serial console (optional)

In another terminal:

```
tio -b 115200 /dev/ttyACM0
```

OpenOCD never uses this port, so it can stay open all the time.

### Step 4 — reset the board

```
$OPENOCD -s $CFG -f $CFG/openocd-jlink-rpi3.cfg -f $CFG/reset.cfg
```

- `-s $CFG -f $CFG/openocd-jlink-rpi3.cfg` — the J-Link and the board description (4 Cortex-A53 cores, JTAG at 1000 kHz). The file only describes the hardware; it does not connect by itself.
- `-f $CFG/reset.cfg` — the commands, one per line:

```
init
targets bcm2837.cpu0; halt
mww 0x3f100024 0x5a000001
```

```
mww 0x3f10001c 0x5a000020
shutdown
```

- `init` — connect to the board through the probe. Every command after it needs it first.
- `targets bcm2837.cpu0; halt` — talk to core 0, and stop it.
- `mww 0x3f100024 0x5a000001` — set the watchdog timer to (almost) zero.
- `mww 0x3f10001c 0x5a000020` — tell the watchdog to reset the chip when it expires. The board reboots.
- `shutdown` — OpenOCD exits.

The `Error: Invalid ACK (0) in DAP response` lines at the end are expected: the chip resets while the probe is still talking to it.

### Step 5 — wait for the board to boot

```
sleep 12
```

### Step 6 — load the test and start it

```
$OPENOCD -s $CFG -f $CFG/openocd-jlink-rpi3.cfg -c "set ELF $ELF" -f $CFG/run-test.cfg
```

OpenOCD reads its options in order: first the board description, then `set ELF …` (the test to load, used by `run-test.cfg` as `$ELF`), then `run-test.cfg`, which holds the commands, one per line:

```
init
targets bcm2837.cpu0; halt
targets bcm2837.cpu1; halt
targets bcm2837.cpu2; halt
targets bcm2837.cpu3; halt
targets bcm2837.cpu0; arm semihosting enable
targets bcm2837.cpu1; arm semihosting enable
targets bcm2837.cpu2; arm semihosting enable
targets bcm2837.cpu3; arm semihosting enable
targets bcm2837.cpu0; load_image $ELF
targets bcm2837.cpu0; reg cpsr 0x600001da; resume 0x1003c
targets bcm2837.cpu1; resume
targets bcm2837.cpu2; resume
targets bcm2837.cpu3; resume
```

What each part does:

1. **Connect** (`init`).
2. **Stop the 4 cores** (`targets … halt`, for cores 0–3), so nothing runs while the program is copied.
3. **Turn semihosting on for each core.** The test prints by stopping the core for a moment; OpenOCD sees the stop, prints the text and lets the core go on. Every core needs it, because any core can print once it runs.
4. **Copy the program into RAM** (`load_image $ELF`, through core 0).
5. **Start core 0** at the program's entry:
    - `reg cpsr 0x600001da` — put the core in 32-bit mode: the low bits `0x1a` select HYP (hypervisor) mode, and `0x1c0` masks the aborts, IRQs and FIQs;
    - `resume 0x1003c` — run from the entry point.
6. **Let cores 1–3 go on where they were** (`resume` with no address): back in the firmware's loop, waiting on their mailbox 3.

Core 0 starts the system. When the kernel is ready, the test wakes each of cores 1–3 (`release_one()` in `test/boards/rpi-zero-2w/src/smp.cpp`): it first writes the core's start address into its `__smp_spin` slot, then writes `_start` into the core's mailbox 3. The core leaves the firmware's loop, runs `_start`, and finds its `__smp_spin` slot

already written — so whatever the previous program left in RAM there does not matter, and OpenOCD does not have to clear it.

this can be simplified by tcl script :

```
# Connect to the board.
init

# Halt all four cores and enable semihosting.
foreach core {0 1 2 3} {
    targets bcm2837.cpu$core
    halt
    arm semihosting enable
}

# Load the ELF once, using core 0.
targets bcm2837.cpu0
load_image $ELF

# Configure the initial execution state and start core 0.
reg cpsr 0x600001da
resume 0x1003c

# Start the other cores.
foreach core {1 2 3} {
    targets bcm2837.cpu$core
    resume
}
```

### Step 7 — watch the result

The test's messages appear in this OpenOCD window. It passed when it prints `RESULT: PASS`. Then press **Ctrl-C** to stop OpenOCD. An SMP test shows that cores 1–3 started, e.g. `join: c1=3 c2=3 c3=3`.

How long to wait at most: `smp_test0`–`smp_test3` 120 s; `smp_test4`, `usb_test`, `cmsis-os-validator`, `mutex-stress`, `rtos-apis` 300 s; `sd_test` 450 s; `smp-num-test`, `smp-pipeline-test`, `smp-pro-cons-test` 600 s; `smp-mat-test`, `smp-mat-sdcard-test` 900 s. Printing is slow over JTAG, so the chatty tests take long.

### Step 8 — the next test

Go back to step 2: the reset of step 4 puts cores 1–3 back in the firmware's loop. If a test hung, power-cycle the board first.

---

## AArch64 (64-bit port)

The same steps; only these things change:

| | **AArch32** | **AArch64** |
|---|---|---|
| SD card | `arm_64bit=0` | `arm_64bit=1` |
| Config folder `CFG` | `$A32/test/boards/rpi-zero-2w` | `$A64/test/boards/rpi-zero-2w` |
| Entry point | `0x1003c` | `0x80000` |
| Start core 0 | `reg cpsr 0x600001da`, `resume 0x1003c` | `reg pc 0x80000`, `resume` |
| Cores 1–3 | `resume` where they are | `resume` where they are |
| JTAG speed | 1000 kHz | 4000 kHz |

### Step 2 (AArch64)

```
D=$BUILD/aarch64-rpi-zero-2w-cmake-gcc-debug/platform-bin/port-tests/test
CFG=$A64/test/boards/rpi-zero-2w
ELF=$D/smp_test0-hwd
```

Nothing to read from the ELF: the entry is always `0x80000`.

### Step 4 (AArch64) — reset the board

```
$OPENOCD -s $CFG -f $CFG/openocd-jlink-rpi3.cfg -f $CFG/reset.cfg
```

The same as for AArch32, with the 64-bit port's config folder (its `reset.cfg` has the same lines).

### Step 5 (AArch64)

```
sleep 12
```

### Step 6 (AArch64) — load the test and start it

```
$OPENOCD -s $CFG -f $CFG/openocd-jlink-rpi3.cfg -c "set ELF $ELF" -f $CFG/run-test.cfg
```

The 64-bit port's `run-test.cfg` differs from the 32-bit one in one line, the start of core 0:

```
targets bcm2837.cpu0; reg pc 0x80000; reg pc; resume
```

The same parts as for AArch32, except:

- **Start core 0**: `reg pc 0x80000` sets its program counter to the entry, `reg pc` reads it back (OpenOCD prints `pc (/64): 0x0000000000080000`, a check only), `resume` runs it. A 64-bit core keeps the mode it was stopped in (EL2), and the startup code reads that mode, so there is no CPSR to set.
- **Cores 1–3** wait in the firmware's loop on their release word (`0xd8 + 8*core`), not on a mailbox; the test writes `_start` there after it has written their `__smp_spin` slot.

Steps 3, 7 and 8 are the same as for AArch32.

---

## With the Olimex ARM-USB-OCD instead of the J-Link

In steps 4 and 6, write `openocd-olimex.cfg` instead of `openocd-jlink-rpi3.cfg`. Nothing else changes.

## With a CMSIS-DAP probe instead of the J-Link

A Raspberry Pi Debug Probe, or a Pico running the debugprobe firmware, on the same JTAG pins: in steps 4 and 6, write `openocd-cmsisdap-rpi3.cfg` instead of `openocd-jlink-rpi3.cfg` (with `hw.sh`: `PROBE=cmsis-dap`). Leave the probe's reset line off the Pi's RUN pad: held low, it keeps the Pi in reset and the JTAG chain reads all zeroes. With two CMSIS-DAP probes connected, OpenOCD takes the first one it finds; add `-c "adapter serial <serial>"` before the config.

## If something goes wrong

- **`LIBUSB_ERROR_BUSY`** — another OpenOCD still holds the probe: `pkill -9 -f openocd`.

- **`Invalid ACK (0) in DAP response` in step 6**, then the cores cannot be examined — the JTAG link dropped, not the test. Lower the speed: add `-c "adapter speed 1000"` right after the config file, and check the board's power supply.
- **The cores halt in the wrong state** (AArch32 card on an AArch64 test, or the other way round: `ARM state … Hypervisor` instead of `AArch64 … EL2H`) — the SD card boots the other width, and the test times out; see "The SD card".
- **Nothing prints** — power-cycle the board and start again from step 4.